ICT133

Timed Online Assignment – July Semester 2021

Structured Programming

Thursday, 18 November 2021 **1:00 pm – 3:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **EIGHT (8)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, Your Name, and SUSS PI No. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAXYZ_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

Question 1

Express a sequence of statements based on computational logic.

(a) Write statements for the following tasks:

(i) Display the message `Group A` only if a given whole number, `n`, ends with 2, 5 or 7. Do not convert `n` into a string.

(3 marks)

(ii) Display the message `Valid` if a given string `s`, has exactly 7 characters and the first character is either `A, B` or `C,` all in uppercase. Display the message `Invalid` otherwise.

(4 marks)

(iii) Display the message `Group A` if a string contains at least one letter A, either in uppercase or lowercase in the last 3 positions of the string. Display `Group B` if a string contains the letter B, either in uppercase or lowercase in all last three positions. Display the message `Group C` otherwise. Do not use loop/repetition structure. Use the slice operator.

(5 marks)

(b) Given a string, `s` that contains a sequence of digits, e.g., '1254856439'.

(i) Write a code segment that prints the odd digits of the string, on a single line, and separated by a space. For the example string, the output is `1 5 5 3 9.`

(5 marks)

(ii) Write a code segment that processes the digits in the string in reverse order. The code segment determines whether a digit is smaller than both its immediate left and right neighbours, and displays the digit if it is smaller than both its neighbours. Do not display the digits at both ends of the sequence as these digits have only one neighbour. For the example string, the output is `3 5.`

(8 marks)

Question 2

This question assumes there are two collections: a collection that contains student numbers, names and scores of students who fail ICT133, and another collection that contains student numbers, names and scores of students who fail ICT162. You are to propose both control structure as well as structure for the collection for tasks 1 to 3.

You are to propose structure for the collection for tasks 1 to 3 in Q2(a), and then based on your proposed collection structure in Q2(a), propose the control structure for Q2(b).

Task 1
Express and determine how many more students fail ICT133 than students who fail ICT162. If there are fewer students failing ICT133 than ICT162, the number is negative.

Task 2
Determine whether there are students who fail both ICT133 and ICT162.

Task 3
Determine the average score of students who fail ICT133 and the average score of students who fail ICT162.

(a) Use and apply your understanding of how data structures store and process information.
- Propose a structure for the collection for each task. The structure may be different for a different task.
- Justify the structure you propose for each task based on the required operations for each task.

(12 marks)

(b) Use your understanding of the building blocks of computers and programs, and determine which Python control structures apply to tasks 1 to 3:
- `if/if-else` selection and
- `for/while` repetition.

For each task, explain your choices of control structures as well as how the control structures are applied in the logic for performing the task, based on the collection structures proposed in Q2(a).

Note that more than one control structures may be required for a task, but you are restricted to using each control structure to a maximum of 2 tasks of tasks 1 to 3. Using built-in functions such as sum does not add to the count of each of the two listed control structures used. Also, note that list comprehension is considered as a repetition structure. Furthermore, a list comprehension that includes `if` is considered as a repetition structure and a selection structure.

(13 marks)

Question 3

Employ structured programming principles to develop a program. Do NOT use global variables.

(a) Implement a function, **`allSameValuesInSequence`** which has one parameter: a sequence of values. The function returns True if all values in the sequence are the same, and False otherwise.

(5 marks)

(b) Implement a function, **`allDifferentValuesInSequence`** which has one parameter: a sequence of values. The function returns True if all values in the sequence are different, and False otherwise.

(6 marks)

(c) Write a function, **`readNumbers`** which returns a sequence of whole numbers that a user enters as input. The user hits the ENTER key when he wishes to end the input session, and the function allows the user to end the input session if the user has entered at least 2 numbers. Refer to the example run.

(6 marks)

(d) Write a function, **`main`** which starts by calling the function **`readNumbers`**, and then repeatedly allows the user to select one of the choices in the menu:

```
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
```

The choice is then processed as shown in the sample run (input underlined).

Sample run:

```
Enter a number:
Please enter at least two numbers before ending input session
Enter a number: 1
Enter a number:
Please enter at least two numbers before ending input session
Enter a number: 2
Enter a number:
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
Enter choice: 2
Not all numbers are same in [1, 2].
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
Enter choice: 3
All numbers are different in [1, 2].
```

```
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
Enter choice: 1
Enter a number: 2
Enter a number: 2
Enter a number: 2
Enter a number:
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
Enter choice: 2
All numbers are same in [2, 2, 2].
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
Enter choice: 3
Not all numbers are different in [2, 2, 2].
Menu
1. Read another number sequence
2. Determine all numbers same
3. Determine all numbers different
0. Exit
Enter choice: 0
Program ends
```

(8 marks)

Question 4

Solve computational problems using structured programming. Do NOT use global variables.

(a) The text file, sales.txt contains product sales data, as shown here:

```
pencil 2
lead 2
pen 1
lead 4
notebook 3
pencil 4
lead 1
```

Each line in the file contains a product name and the quantity sold. For example, the first line records 2 units of pencil sold.

Another text file, salesToDate.txt contains the cumulative product sales, as shown here:

```
pencil 27
notebook 18
```

Each line in the file contains a product with the quantity sold accumulated. In this example, only two types of products have been sold previously, the total quantity sold for pencil is 27 and the total quantity sold for notebook is 18.

Develop and write a structured program (with functions) to update the file, salesToDate.txt. You are to add the data in sales.txt to the corresponding product in salesToDate.txt.
The final content in salesToDate.txt based on the example data, is shown here:

```
pencil 33
notebook 21
lead 7
pen 1
```

- There are two lines for pencil in sales.txt.
  ```
  pencil 2
  pencil 4
  ```
  Thus, the new accumulated total for pencil is 27 (old accumulated) + 2 + 4 = 33.

- There were no entries for lead and pen in the original file, salesToDate.txt. Therefore, new entries are added for these two products to salesToDate.txt.

(15 marks)

(b) Given a dictionary with product names as keys and prices as values:

```
prices = {'pencil': 1.85, 'pen': 1.90, 'notebook': 0.90, 'lead': 2.50}
```

(i) Write a function **`getOneSale`** that has one parameter: the dictionary, `prices`. It returns a dictionary for one sale., e,g,, `{'pencil': 6, 'pen': 8, 'notebook': 4}`. The sale consists of product names as keys and quantity sold as values.

Use the package random to make a sale dictionary.

- For each product, randomly choose either a yes or a no. If the choice is yes, the product is included in the sale.
- For each product included in the sale, randomly select a number between 1 and 15 for quantity sold.
- The name of the included products and their quantity sold are added as entries in the sale dictionary.

(ii) Write a function that accepts the dictionaries, `sale` and `prices`, and prints a receipt for the sale. Ensure proper alignment using formatted print. An example is shown here:

```
14 X pencil     = $25.90
03 X pen        = $ 5.70
12 X lead       = $30.00
Total           = $61.60
```

(10 marks)

----- END OF PAPER -----